
Name

tkwintrack — track and outline windows in a Tk GUI, and display their pathname

Synopsis

```
package require tkwintrack
```

Description

This reference page corresponds to tkwintrack version 2.1

Tkwintrack is an inspection tool for developers of Tk GUI's. It extends the interpreter of a Tk application with the machinery to track the individual windows (widgets) in a GUI using the mouse pointer. Upon detection of a window, its outline is highlighted and its pathname is displayed in a small separate widget: the *pathname widget*. Window pathnames displayed in the pathname widget may be copied to the clipboard using a keyboard shortcut.

After loading the package into the client application, tkwintrack starts in configuration mode: a dialog is displayed, which lets the user adjust appearance and behaviour of the package's functionality (see also the [the section “Configuration mode”](#)). After clicking the Continue button in this dialog, tkwintrack enters detection mode.

Detection mode

In detection mode, the pathname widget appears on screen and the user may track windows using the mouse pointer. While moving the pointer around inside the GUI, individual windows are detected, their outline highlighted, and their pathname displayed in the pathname widget.

The following mouse button actions are available when the mouse pointer is over the pathname widget:

- right-clicking opens the configuration dialog, suspending detection mode.
- pressing and holding the left mouse button allows dragging of the pathname widget.

The following functionalities are accessible through keyboard shortcuts, irrespective of the position of the mouse pointer or the pathname widget:

- Toggling detection mode on/off. Toggling detection mode off withdraws the pathname widget and the highlighted window outlines from the screen, thus letting the client GUI regain its unaffected appearance. Default keyboard shortcut is **Alt+7**.
- Copying the pathname of the currently detected window to the clipboard. Default keyboard shortcut is **Alt+8**.
- Recalling the pathname widget. This is useful if the pathname widget was configured to not follow the mouse pointer, and the pathname widget was dragged away from the active area of the client application (or conversely). In this situation, the pathname widget can't be reached anymore with the mouse pointer unless a window in the client application is moved towards the pathname widget. The keyboard shortcut obviates such an awkward action: pressing it moves the pathname widget from anywhere on the screen to the location of the mouse pointer. Default keyboard shortcut is **Alt+9**.

To reduce the probability of interfering with the bindings of the client application, rather uncommon key sequences have been chosen for the default keyboard shortcuts. See [the section “Configuration mode”](#) for how to configure them differently.

Configuration mode

Clicking the right mouse button when the mouse pointer is over the pathname widget, displays the configuration dialog. By means of this dialog the user can adjust appearance and behaviour of various functionalities of tkwintrack.

Tkwintrack remembers the settings for up to 10 different client applications. Upon loading the package in a client application, tkwintrack automatically applies the settings as they were in the previous session for that particular client application. If the limit of 10 client applications is exceeded, tkwintrack discards the oldest settings.

The available options are, by category:

Outline highlighting

- Outline color
The color used for the outline drawn around a tracked window. Default is red.
- Outline width
The width of the outline drawn around a tracked window. Default is 2 pixels.

Pathname widget

- Background color
The background color (fill) of the pathname widget. Default is light yellow.
- Foreground color
The color of the text and border of the pathname widget. Default is black.
- Follow pointer
This setting determines whether the pathname widget follows the mouse pointer when a new window is detected. Default is on. Changing this setting has only effect in the current session. New sessions always have this setting on.

Keyboard shortcuts

Moving the mouse pointer over the value for a keyboard shortcut activates key selection mode, and the key sequence becomes highlighted. While it is highlighted, the user can define a new key sequence for the shortcut by pressing keys. The shortcut may exist of a single regular key, optionally preceded by one or two modifier keys (`Control`, `Shift`, `Alt`, `Meta`, `Mod1` through `Mod5`). As soon as a valid key sequence is detected, it is accepted and the key selection mode is deactivated.

Exit, Reset and Continue buttons

The lowest row in the configuration dialog contains three buttons:

Exit

Exits tkwintrack and removes the package from the client interpreter.

Reset

Reverts all configurable settings to their default values.

Continue

Enters detection mode, using the newly configured settings.

Requirements

Tk version

Tkwintrack requires Tk 8.5 or newer.

Windetect version

Tkwintrack requires windetect 2.0 or newer.

Windowing system / Operating system

Tkwintrack functions overall well with the X11 windowing system (tested on Linux with various window managers and desktop environments), and with the windowing system supplied by the Microsoft Windows operating system. Proper functioning has not been tested with other windowing systems or operating systems, and therefore no claims are made about it.

Limitations**Generic limitations**

Tkwintrack relies on a client application with a regular Tcl/Tk interpreter, offering standard commands and Tk bindings as provided by the Tcl/Tk language. Tkwintrack doesn't cope with situations where standard Tcl/Tk commands were removed or renamed, and it relies on the binding tag `all` being associated with each widget in the client application.

Tkwintrack operates on a single Tk instance in a single interpreter. If a GUI employs multiple Tk instances (in as many distinct interpreters), the user needs to decide for each of them whether to load tkwintrack into it. In order to distinguish multiple instances of tkwintrack, the header of the configuration dialog shows the name of the client interpreter.

Outline widget

The outline is a composed widget consisting of four lines that are drawn just inside a tracked window. When the mouse pointer is right over one of these lines, it is insensitive to the client GUI, which causes the following anomalies:

- the shape of the mouse pointer may be different from what was configured for the tracked window by the client application;
- bindings to mouse events on the tracked window, as defined by the client application, do not work;
- very small windows in the client GUI, positioned at the border of the tracked window (the container window), cannot be detected when they are entirely covered by the outline widget.

For these reasons, it is advised to choose the outline width as small as acceptable.

Programming constructs that reenter the event loop

Programming constructs in tkwintrack that reenter the event loop may interfere with the order of execution of actions scheduled by the client application. During its lifetime, there are several occasions where a running tkwintrack program reenters the event loop. All of them serve to induce a timely screen redraw, either for the outline widget, the pathname widget or the configuration dialog.

The following list enumerates these occasions:

- when the pathname widget is created `update_idletasks` is called. This happens once during the run time of the program, at program initialization.
- each time when the configuration dialog is (re)displayed, a call to `update_idletasks` is made.
- each time when an invalid keyboard shortcut is entered in the configuration dialog, `update_idletasks` is called.
- each time when the pathname is copied to the clipboard `update_idletasks` is called.

- each time when a new window is detected `update_idletasks` is called. This is the only case that occurs while operating the pointer device to track windows.

Test script

Tkwintrack comes with a test script `tkwintrack.test`. There are two aspects to be aware of when running the tests:

- No windows from other applications should occupy the screen area where the test widgets are displayed.
- Quite some tests exercise a setup where toplevel windows overlap on the screen. These tests cannot be usefully run with tiling window managers. Unless the test option `-nowm` was provided to exclude these tests, the test script aborts if it detects a tiling window manager. Even when using the option `-nowm`, some tiling window managers restrict and enforce screen operations to an extent that the test script can't handle them.

The test script is invoked as follows:

```
tclsh tkwintrack.test ?option value? ?option value? ...
```

The command line supports two types of option:

- a. configuration options for the Tcl test harness. These are passed on to `tcltest`. Please see the documentation for the package `tcltest` for details.
- b. one option, specific to `tkwintrack`, that modifies the test script behaviour.

-eyefriendly

This flag makes the test script pause one second at appropriate times so that human vision can follow the changes to the screen.

-nowm

Presence of this flag on the command line makes the test script exclude those tests, where the window manager is involved in bringing about the detection events.

Author

Erik Leunissen

See also

`bind(n)`, `bindtags(n)`, `tcltest(n)`, `update(n)`, `windetect(n)`

License

Copyright © 2020 Erik Leunissen.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at:

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.